CNN Image Classification with Grad-CAM

Done by:Albatul Altoum

Introduction

This project investigates the application of Convolutional Neural Networks (CNNs) for image classification using the CIFAR-10 dataset. Beyond achieving high classification accuracy, the goal is to interpret and visualize the model's decision-making process through Grad-CAM. I focus on implementing a custom CNN model, comparing its performance to a pretrained ResNet18, and analyzing how architectural choices impact model behavior.

1.Model Implementation and Comparison

Basic CNN

I first implemented a basic CNN model from scratch consisting of a few convolutional and pooling layers.
This simple architecture allowed us to understand the fundamental operations of convolutional networks on CIFAR-10.

VGG-Style CNN

After evaluating the basic CNN, I designed and implemented a VGG-style CNN model.
The model is composed of multiple convolutional layers with ReLU activation functions, followed by MaxPooling layers and Dropout for regularization.
Fully connected layers are used at the end for classification.

Pretrained ResNet18

I also used a pretrained ResNet18 model, where only the final fully connected layer was replaced to adapt the model to CIFAR-10's 10 classes.

The rest of the model was fine-tuned to allow some layers to adjust to the new dataset.

Architectural Comparison

I compared the performance of three models: the Basic CNN, the VGG-Style CNN, and the pretrained ResNet18.
The basic CNN provided a good starting point but was limited by its shallow depth.
The VGG-style CNN improved feature extraction through deeper layers, and the ResNet18 further enhanced performance using residual connections.

2. Data Augmentation and Training Setup

Data Preprocessing

To enhance the training data and improve model generalization, the following data augmentation techniques were applied:

- RandomHorizontalFlip to introduce invariance to object orientation.
- RandomCrop with padding of 4 to simulate small translations.
- Normalization with mean = [0.5] and std = [0.5] to stabilize training.

Hyperparameters

- **Optimizer:** Adam, chosen for its adaptive learning rate and fast convergence.
- **Learning Rate:** 0.001, providing a balance between convergence speed and stability.
- **Batch Size**: 32, suitable for memory limitations and maintaining good gradient estimates.
- **Epochs:** 10, allowing enough training time without overfitting.

Cross-Entropy Loss was used as the objective function, which is standard for multi-class classification problems.

3. Importance of Convolution, Pooling, and Deep Architectures

Convolutions are critical for automatically learning spatial hierarchies of features in images. They allow the model to detect patterns such as edges, textures, and shapes without manual feature engineering.

Pooling layers like MaxPooling reduce spatial dimensions, making the model computationally efficient and more robust to small translations and distortions in input images.

Deep architectures (e.g., ResNet) are essential for learning highly abstract and complex representations. As networks grow deeper, they capture increasingly sophisticated patterns, but they require careful design (such as skip connections) to avoid problems like vanishing gradients.

Together, these components form the backbone of modern deep learning models in computer vision.

4. Evaluation Metrics

All three models were evaluated using:

- Accuracy: Overall correctness of the predictions.
- Precision: Correctness among predicted positives.
- Recall: Abilityto capture all actual positives.
- F1-Score: Harmonic mean of precision and recall.

Model	Accuracy	Precision	Recall	F1-Score
Basic CNN	0.68	0.68	0.68	0.68
VGG-Style CNN	0.73	0.73	0.73	0.73
ResNet18	0.84	0.84	0.84	0.84

5. Visualization Using Grad-CAM

To interpret the models' focus during classification, I applied Grad-CAM visualization techniques on ResNet18. Grad-CAM generates heatmaps highlighting the regions of the input image that contributed most to the final prediction.

These visualizations provided valuable insights, showing that successful classifications often corresponded to the model focusing on meaningful image regions (such as the face of an animal or the shape of an object).



Figure 1: Grad-CAM visualization showing the model's focus areas during prediction.

6. Misclassified Examples

By examining misclassified samples, I identified common failure cases:

- Confusions between visually similar classes (e.g., cat vs. dog).
- Incorrect focus on background regions rather than the object.

This analysis suggested areas for potential improvements, such as more aggressive augmentation or specialized fine-tuning strategies.



Figure 2: Examples of misclassified images with their true and predicted labels.

7. Insights and Conclusion

While the basic CNN achieved reasonable performance, it was significantly outperformed by deeper architectures such as the VGG-style CNN and ResNet18.

The pretrained ResNet18 significantly outperformed the custom VGG-style CNN in both accuracy and convergence speed. Grad-CAM visualizations confirmed that the models often based their decisions on the correct parts of the images.

Understanding misclassifications provided guidance for future improvements. Convolution, pooling, and deep architectures proved essential for achieving high performance in image classification tasks.

.References

• CIFAR-10 Dataset

• PyTorch Documentation

• Selvaraju et al., Grad-CAM

• Goodfellow et al., Deep Learning (2016)